

Contents

1 How Cameras Work *3*

1.1 The Light That Shines On the Cow *3*

1.2 Light Hits the Cow *5*

1.3 Pinhole camera *6*

1.4 Lenses *7*

1.5 Sensors *9*

1.6 Aspect Ratio *10*

1.6.1 Printing *11*

2 How Eyes Work *13*

2.1 Eye problems *14*

2.1.1 Glaucoma *14*

2.1.2 Cataracts *15*

2.1.3 Nearsightedness, farsightedness, and astigmatism *15*

2.2 Seeing colors *16*

2.3 Pigments *18*

3 Images in Python *21*

3.1 Adding color *22*

3.2 Using an existing image *25*

4 Representing Natural Numbers *27*

4.1 Base-10 (Decimal) *27*

4.2 Base-2 (Binary) *28*

4.2.1 Bits and Bytes 29
4.3 Base-16 (Hexadecimal) 30
4.3.1 Hex Color Codes 31
4.4 Base-8 (Octal) 31
4.4.1 Why Octal Matters 32

5 Floating Point Arithmetic 33
5.1 Why Computers Can't Count Perfectly 33
5.1.1 The Number Line is Continuous and Unbounded 33
5.2 Binary and the Floating-Point System 34
5.2.1 Binary System 35
5.2.2 Floating-Point Number 35
5.2.3 Floating-Point System 37
5.2.4 Connecting to Binary Floating-Point 38
5.3 The IEEE 754 Standard 39
5.3.1 Floats 39
5.3.2 Machine Epsilon 40

A Answers to Exercises 43

Index 47

Chapter 1

How Cameras Work

Let's say it is a sunny day and you are standing in a field a few meters from a cow. You use the camera on your phone to take a picture of the cow. How does that whole process work?

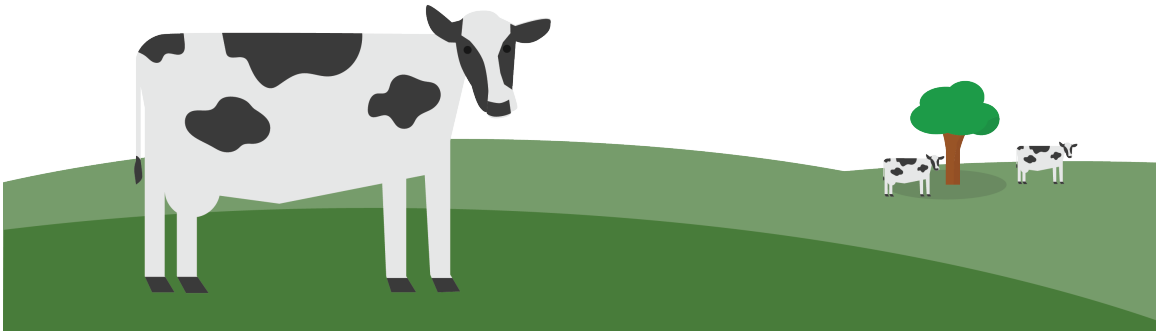


Figure 1.1: A cow in a field.

1.1 The Light That Shines On the Cow

The sun is a sphere of hot gas. About 70% of the gas is hydrogen. About 28% is helium. There's also a little carbon, nitrogen, and oxygen.

Gradually, the sun is converting hydrogen into helium through a process known as "nu-

clear fusion" (which we will be discussing more in a future chapter). A large amount of heat is created in this process. This heat makes the gases glow.

How does heat make things glow? The heat pushes the electrons into higher orbitals. When they come back down to a lower orbital, they release a photon of energy, which travels away from the atom as an electromagnetic wave. Recall that when we talked about reflections, we established that light is a constant stream of particles we call photons. These photons travel at the speed of light.

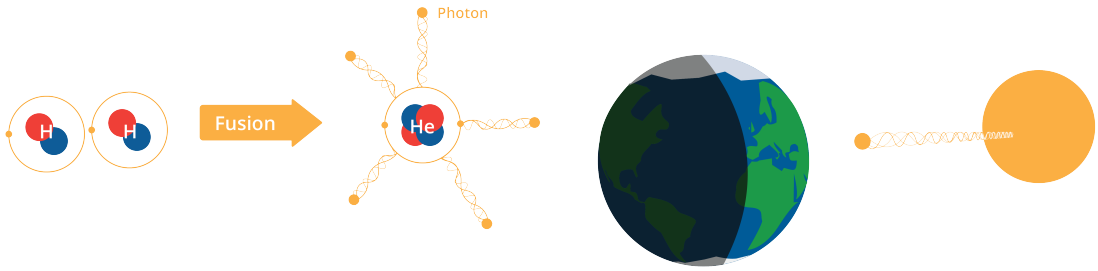


Figure 1.2: Photons are released when the sunlight hits the cow.

Heat is not the only way to push the electrons into a higher orbital. For example, a fluorescent lightbulb is filled with gas. When we pass electricity through the gas, its electrons are moved to a higher orbital. When they fall back to a lower orbital, light is created.

What is the frequency of the wave that the photon travels on? Depending on what orbital it falls from and how far it falls, the photon created has different amounts of energy. The amount of energy determines the frequency of the electromagnetic wave, and ultimately, its color!

In the sun, there are several kinds of molecules and each has a few different orbitals that the electrons can live in. Thus, the light coming from the sun is made up of electromagnetic waves of many different frequencies.

We can see some of these frequencies as different colors, but some are invisible to humans, such as ultraviolet and infrared.

1.2 Light Hits the Cow

When these photons from the sun hit the cow, the hide and hairs of the cow will absorb some of the photons. These photons will become heat and make the cow feel warm. Some of the photons will not be absorbed – they will leave the cow. When you say "I see the cow," what you are really saying is "I see some photons that were not absorbed by the cow."

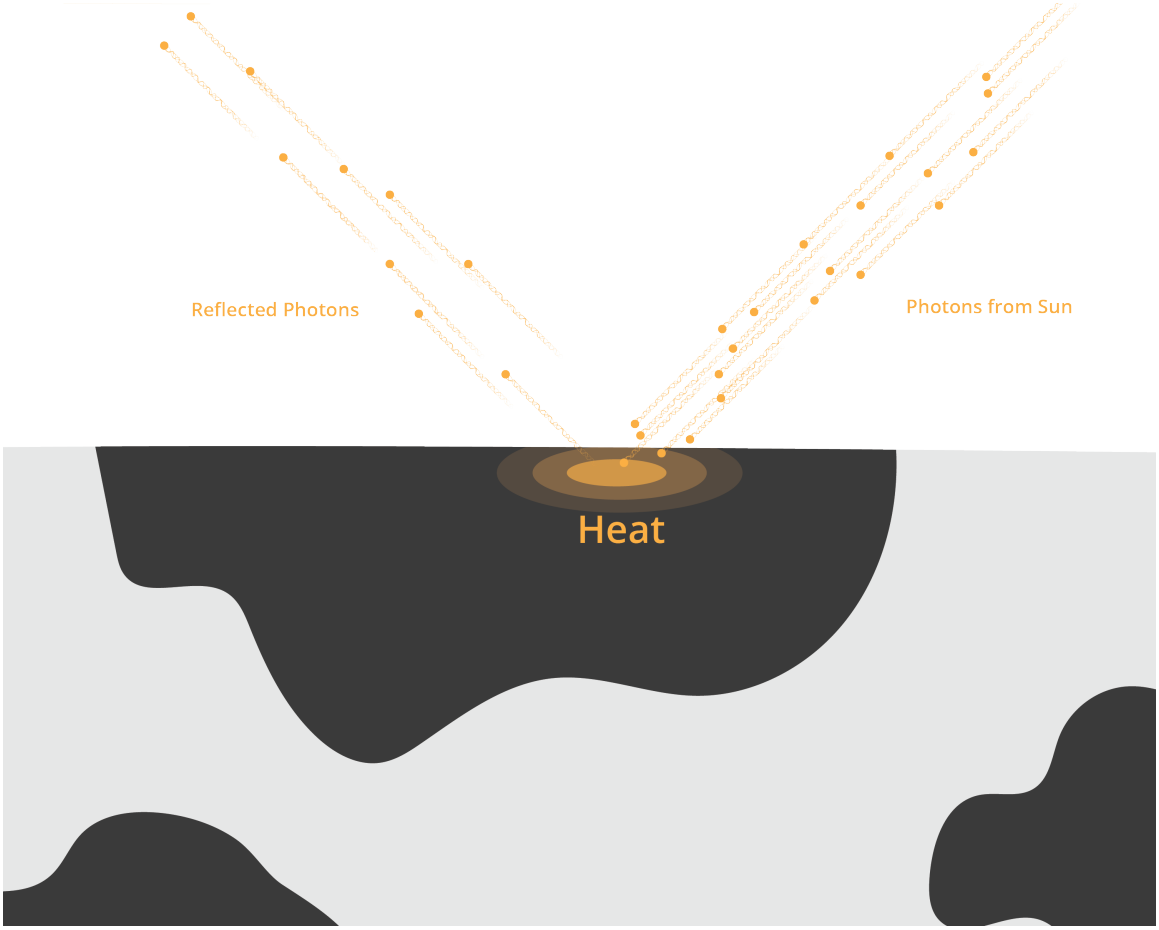


Figure 1.3: Photons hitting the cow create a heated spot, where some are absorbed.

Different materials absorb different amounts of each wavelength. A plant, for example, absorbs a large percentage of all blue and red photons that hit it, but it absorbs only a small percentage of the green photons that hit it. Thus, we say "That plant is green."

White things absorb very small percentages of photons of any visible wavelength. Black things absorb very *large* percentages of photons of any visible wavelength. That is why, on a hot summer day, a black car with black seats and interior will heat up on the inside much hotter than a white car.

Before we go on, let's review: The sun creates photons that travel as electromagnetic waves of assorted wavelengths to the cow. Many of those photons are absorbed, but some are not. Some of those photons that are not absorbed go into the lens of our camera.

1.3 Pinhole camera

The simplest cameras have no lenses. They are just a box. The box has a tiny hole that allows photons to enter. The side of the box opposite the hole is flat and covered with film or some other photo-sensitive material.

The photons entering the box continue in the same direction they were going when they passed through the hole. Thus, the photons that entered from high hit the back wall at a low point. The photons that came from the left hit the back wall on the right. This is how the image is projected onto the back wall, rotated 180 degrees; what was up is down, what was on the left is on the right.

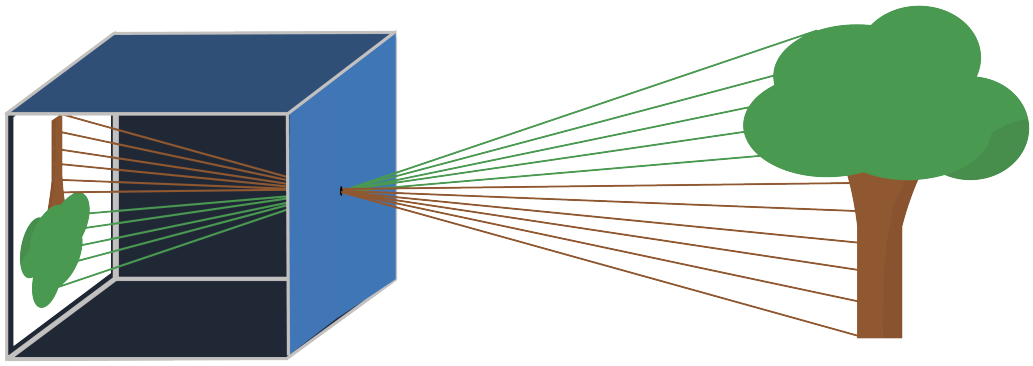


Figure 1.4: A pinhole camera flips the image it "sees" by 180°.

Exercise 1 Height of the image

Let's say that the pinhole is exactly the same height as the shoulder of the cow, and that the shoulder is directly above one hoof. This means the pinhole, the shoulder, and the hoof form a right triangle.

Now, let's say that the camera is being held perpendicular to the ground. The pinhole, the image of the shoulder, and the image of the hoof on the back wall of the camera now also form a right triangle.

These two triangles are similar.

The shoulder is 2 meters from the hoof. The cow is standing 3 meters from the camera. The distance from the pinhole to the back wall of the camera is 3 cm. How tall is the image of the cow on the back wall of the camera?

Working Space

Answer on Page 43

1.4 Lenses

Now, a quick review: A photon leaves the sun in some random direction. It travels 150 million km from the sun and hits a cow. It is not absorbed by the cow, and heads off in a new direction. It passes through the pinhole and hits the back wall of the camera. That seems incredibly improbable, right?

It actually is relatively improbable, especially if there isn't a lot of light — like you are taking the picture at dusk. To increase the odds, we added a *lens* to the camera. If you focus a lens on a wall and you draw a dot on that wall, the lens is designed such that all the photons from the dot that hit the lens get redirected to the same spot on the back wall of the camera — regardless of which path it took to get to the lens.

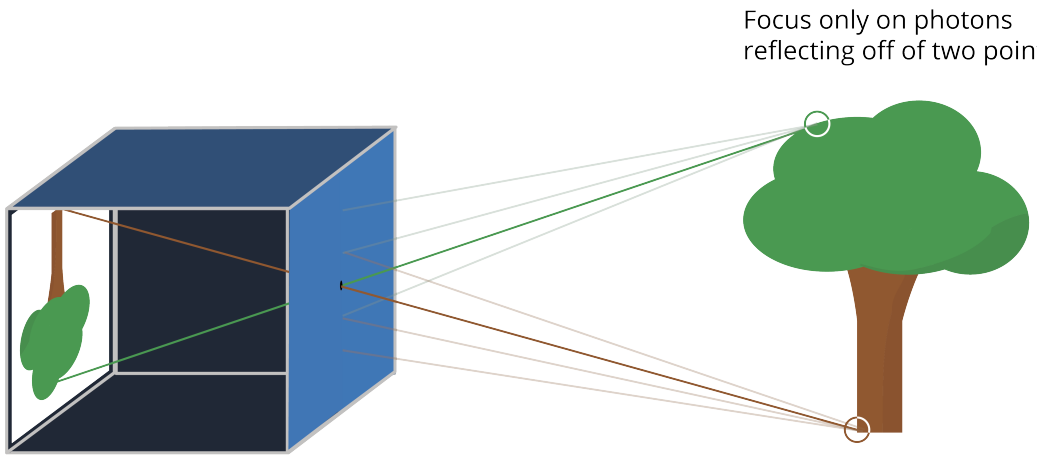


Figure 1.5: A pinhole with a lens allows you to choose the points it reflects.

Note that the image still gets flipped. There is a *focal point* that all the photons pass through.

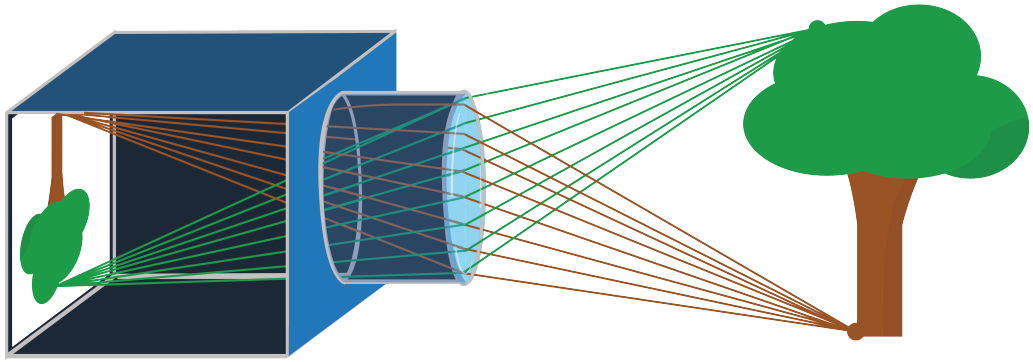


Figure 1.6: A lens still flips the image, but introduces a focal point.

The distance from the lens to its focal point is called the lens's *focal length*. Telephoto lenses, that let you take big pictures of things that are far away, have long focal lengths.

Wide-angle lenses have short focal lengths.

1.5 Sensors

The camera on your phone has a sensor on the back wall of the camera. The sensor is broken up into tiny rectangular regions called pixels. When you say a sensor is 6000 by 4000 pixels (most common ratio for photography), we are saying the sensor is a grid of 24,000,000 pixels: 6000 pixels wide and 4000 pixels tall.

In the sensor, each individual pixel is hit by an incoming photon. If the photon's energy (recall $E = hf$) is less than the Work Function Φ to dislodge an electron, the light simply passes through or reflects, and the pixel remains *dark*, or what we call *black*. If $hf \geq \Phi$, the photon is absorbed, and its energy is transferred to an electron, which *jumps* into the conduction band, appearing like *white* to us. Camera sensors then convert their pixels into a binary mapping, using voltage measurements.

Each pixel has three types of cavities that take in photos. One of the cavities measures the amount of short wavelength light, like blues and violets. One of the cavities measures the long wavelength light, like reds and oranges. One of the cavities measures the intensity of wavelengths in the middle, like greens. Thus, if your camera has a *resolution* of 6000×4000, the image is 24,000,000 pixels yields three numbers: intensity of long wavelength, mid wavelength, and short wavelength light, for a total of 72,000,000 numbers. We call these numbers "RGB" for Red, Green, and Blue. The RGB values range from $0 - 255$ for each channel. We will talk more about this when hexadecimal is introduced.

Exercise 2 Camera Pixels

A digital camera sensor is made of a material with a work function of $\Phi = 1.1$ electronvolts. If the sensor is exposed to monochromatic light with a wavelength of $\lambda = 1200$ nm, what does the resulting image look like? Why?

Hint: $E = \frac{1240}{\lambda}$ where λ is in nanometers and E is in electronvolts (eV).

Working Space

Answer on Page 43

1.6 Aspect Ratio

When you are buying a monitor, TV, or even smartphone, you may hear the term **aspect ratio**. The aspect ratio is a set of two numbers representing the ratio between the width and the height. Conventionally, the width comes first, separated by a colon, and then the height:

$$\text{width} : \text{height}$$

Some of the most common aspect ratios are:

- $16 : 9$ (known as Widescreen)
- $17 : 9$
- $9 : 16$
- $4 : 3$ (known as Fullscreen)
- $4 : 5$
- $3 : 2$
- $1 : 1$ (perfect square, like on a smartwatch)

As we talked about above, the aspect ratio doesn't describe the actual size of the screen or image. Instead, it describes the shape. A movie theatre screen can have the same aspect ratio as your monitor, even if the screen is 360 inches and the monitor is 24 inches.

Aspect ratio describes the proportional relationship between width and height. Resolution describes the total number of pixels used to represent an image.

For example, both of the following resolutions have a $16 : 9$ aspect ratio:

$$1280 \times 720$$

$$1920 \times 1080$$

Mathematically, the aspect ratio is a dimensionless quantity. Since both width and height are measured in the same units (such as centimeters, inches, or pixels), the units cancel when forming the ratio. This means aspect ratio describes proportion rather than physical measurement. Most often, we determine the aspect ratio of a digital display by comparing the number of horizontal pixels to vertical pixels.

For example:

$$1920 \times 1080 \quad \Rightarrow \quad \frac{1920}{1080} = \frac{16}{9} = 16:9$$

Digital cameras and smartphone cameras capture images using a rectangular sensor. The shape of this sensor determines the default aspect ratio of the image. For example:

- Many digital cameras use a $3:2$ sensor.
- Micro Four Thirds cameras use $4:3$.
- Most video recording uses $16:9$.

Important: when you change the aspect ratio setting on a camera, the device does not change the sensor itself. Instead, it crops part of the image to match the selected ratio. Cropping reduces the number of recorded pixels, which may reduce file size. However, actual file size also depends on compression methods used by the device.

Human vision has a wider horizontal field of view than vertical field of view. Because of this, wider aspect ratios such as $16:9$ often feel more natural and immersive. This is one reason modern TVs and cinema formats favor wider screen shapes. However, with the rise of cellphones, we have switched to consistently using portrait or vertical mode, and video aspect ratios are more consistently becoming $9:16$. This allows for consistency in device quality, and commonly used for vertical videos, like in short-form content, compared to traditional widescreen videos, such as Youtube or traditional horizontal video capturing.

1.6.1 Printing

You can imagine this may cause a problem for printing out photos. Most printing paper is 8.5 inches $\times$ 11 inches or 8 inches $\times$ 10 inches. This causes an issue, as that fits a $8.5:11 \iff 17:22$ or $4:5$ aspect ratio, respectively. We combat this by using very standardized printing sizes (in inches):

- $6 \times 4 \to 3:2$,
- $7 \times 5 \to 7:5$,
- $10 \times 8 \to 5:4$,
- and, rarely, $8.5 \times 11 \to 1.29:1$

Going to any framing or printing store will give you a very standardized set of sizes, or some multiple of these.

Most smartphones use $4:3$ or $3:2$, which will work for 4×6 inches or 8×6 inches,

respectively.

Chapter 2

How Eyes Work

Dr. Craig Blackwell has made a great video on the mechanics of the eye. You should watch it: https://youtu.be/Z8asc2SfFHM

Mechanically, your eye works a lot like a camera. The eye is a sphere with two lenses on the front: The outer lens is called the *cornea*, while the second lens is simply called "the lens." Between the two lenses is an aperture that opens wide when there is very little light, and closes very small when there is bright light. The opening is called the *pupil* and the tissue that forms the pupil is called the *iris*. When people talk about the color of your eyes, they are talking about the color of your iris. The blackness at the center of your iris is your pupil.

There are two types of photoreceptor cells in your retina: rods and cones. The rods are more sensitive; in very dark conditions, most of our vision is provided by the rods. The cones are used when there is plenty of light, and they let us see colors.

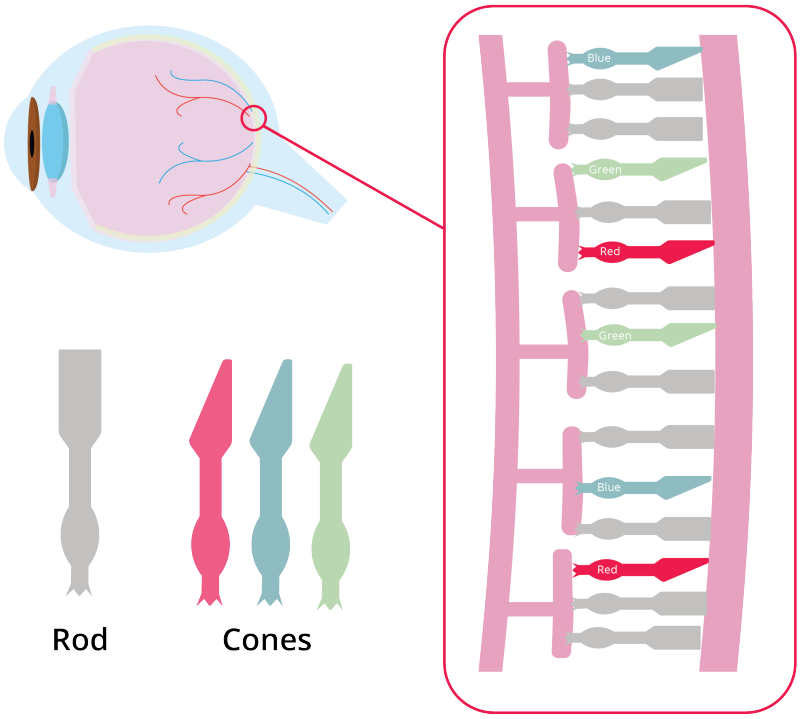


Figure 2.1: The cones and rods of the eye act as sensors and cognitive vision.

The white part around the outside of the eyeball? That is called the *sclera*.

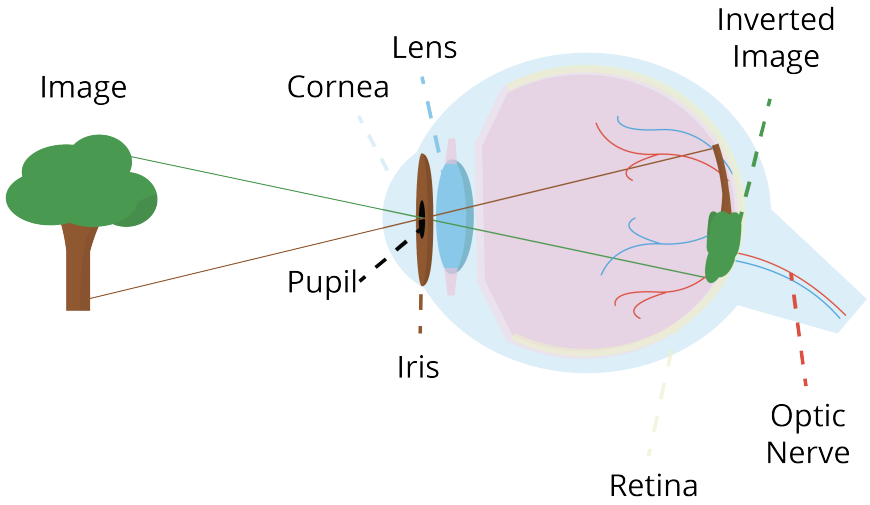


Figure 2.2: The eye works just like a camera's lens.

The walls of the eye are lined inside with the *retina*, which has sensors that pick up the light and send impulses down the optic nerve to your brain.

Just like a camera, the images are flipped when they get projected on the back of the eye (see Figure 2.2).

2.1 Eye problems

Now that you know the mechanics of the eye, let's go over a few things that commonly go wrong with the eye.

2.1.1 Glaucoma

The space between your cornea and lens is filled with a fluid called *aqueous humor*. To feed the cells of the cornea and lens, the aqueous humor carries oxygen and nutrients like blood would, but unlike blood, it is transparent so you can see. Aqueous humor is constantly being pumped into and out of that chamber. If aqueous humor has trouble

exiting, the pressure builds up and can damage the eye. This is known as *glaucoma*. See Figure 2.3.

2.1.2 Cataracts

The lens should be clear. As a person ages, the proteins in the lens break down and clump together, becoming opaque. This can also be accelerated by diabetes, too much exposure to sunlight, obesity, and high blood pressure. From the outside, the eye will look cloudy. This is called a *cataract*, and it makes it difficult for the person to see.

This problem can be corrected, however. The person's cloudy lens is removed and replaced with a clear, manufactured lens.

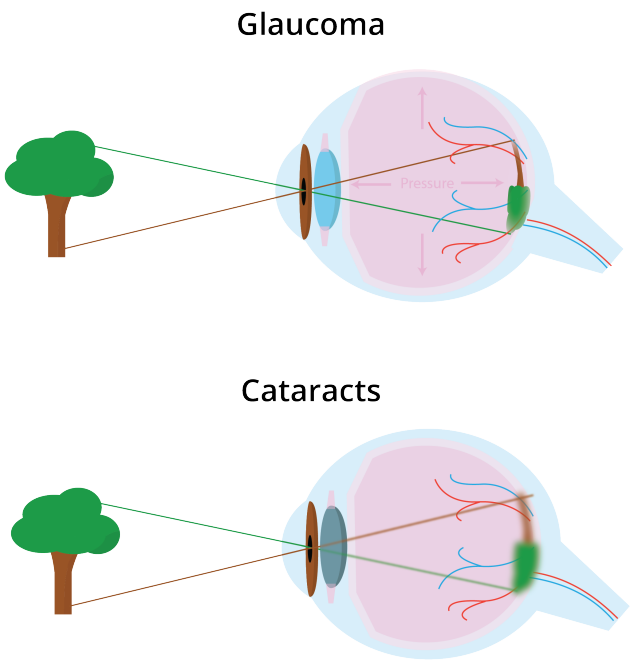


Figure 2.3: Cataracts and Glaucoma represented in the eye cross section.

2.1.3 Nearsightedness, farsightedness, and astigmatism

If you are in a dark room and a tiny LED is turned on, the photons from that LED can pass through your cornea in many different places. If your eye is focusing on that light correctly, all the photons should meet up at the same place on the retina.

FIXME: Diagram here

If the lenses are bending the light too much, the photons meet up before they hit the retina and get smeared a bit across it. To the person, the LED would appear blurry. The eye is said to be *nearsighted* or *myopic*. This signals that near objects appear correctly, but farther objects appear blurry.

If the lenses are not bending it enough, the photons would meet up behind the retina. Once again, they get smeared a bit across the retina and the LED looks blurry to the person. The eye is said to be *farsighted* or *hyperopic*. The objects which are distant can appear clearly, while closer objects are heavily blurred.

Your lenses are supposed to bend the photons the same amount vertically and horizontally. If one dimension is focused, but the other is myopic or hyperopic, the eye is said to have *astigmatism*.

Myopia, hyperopia, and astigmatism can be corrected with glasses or contact lenses. Doctors can also do surgical corrections, usually by changing the shape of the cornea.

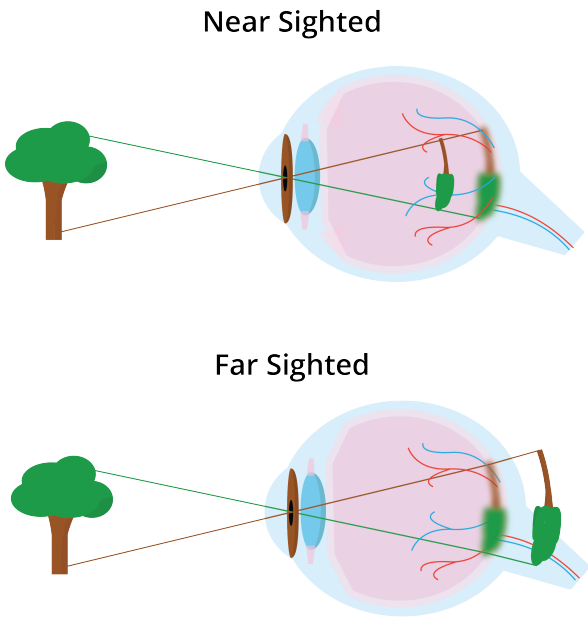


Figure 2.4: Nearsightedness versus farsightedness.

2.2 Seeing colors

TED-Ed has made a good video on how we see color. Watch it here: https://youtu.be/l8_fZPHasdo

When a rainbow forms, you are seeing different wavelengths separating from each other.

In the rainbow:

- Red is about 650 nm.
- Orange is about 600 nm.
- Yellow is about 580 nm.
- Green is about 550 nm.
- Cyan is about 500 nm.
- Blue is about 450 nm.
- Violet is about 400 nm.

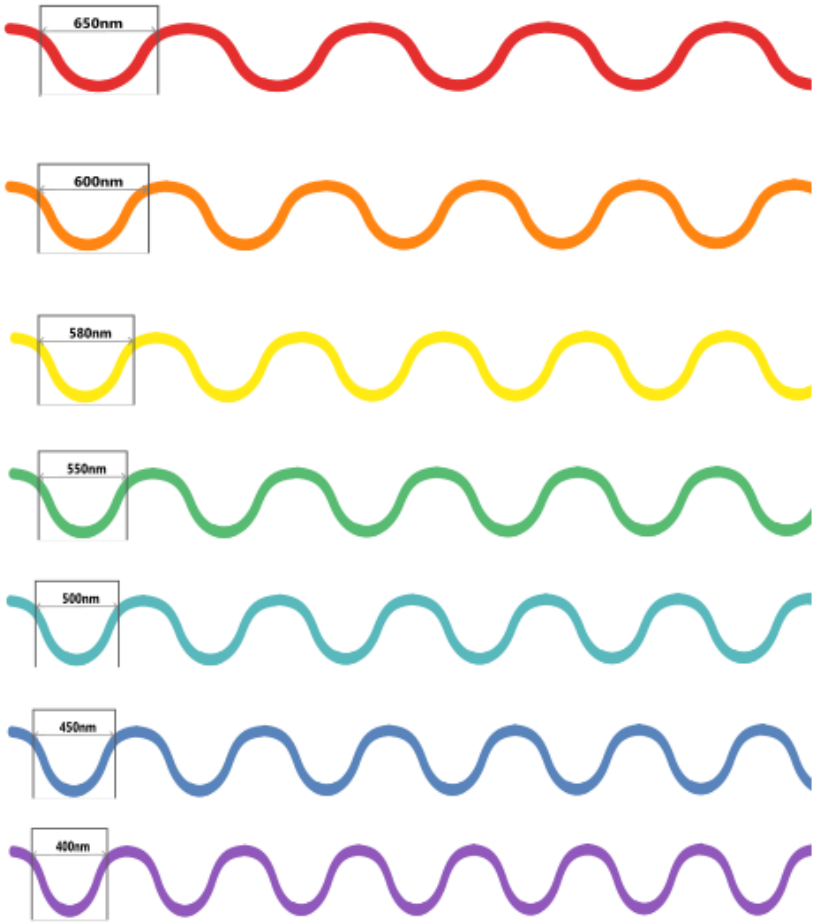


Figure 2.5: All colors have different wavelengths.

If you shine a light with a wavelength of 580 nm on a white piece of paper, you will see yellow.

However, if you shine two lights with wavelengths of 650 nm (red) and 550 nm (green), you will also see yellow.

Why? Our ears can hear two different frequencies at the same time. Why can't our eyes see two colors in the same place?

As mentioned above, the cone photoreceptors in our eyes let us see colors. There are three kinds of cones:

- Red: Cones that let us see the frequencies up to about 700nm.
- Green: Cones that are most sensitive to frequencies near 550nm.
- Blue: Cones that are most sensitive to frequencies near 450nm.

When a wavelength of 580 nm hits your retina, it excites the red and green receptors, and your brain interprets that mix as yellow.

Similarly, when light that contains both 650 nm and 550 nm waves hits your retina, it excites the red and green receptors, and your brain interprets that mix as yellow.

You can't tell the difference!

Now we know why the sensors on the camera are RGB. The camera is recording the scene as closely as necessary to fool your eye.

A TV or a color computer monitor only has three colors of pixels: red, green, and blue. By controlling the mix of them, it creates the sensation of thousands of colors to your eye.

2.3 Pigments

A color printer works in the opposite fashion. Instead of radiating colors, it puts pigments on the paper that absorb certain frequencies. A pigment that absorbs only frequencies near 650 nm (red) will appear to your eye as cyan. This makes sense, because the sensation of cyan is created when your blue and green receptors are activated.

Thus, pigment colors come in:

- Cyan: absorbs frequencies around red
- Magenta: absorbs frequencies around green

- Yellow: absorbs frequencies around blue

If you buy ink for a color printer, you know there is typically a fourth ink: black. If you put cyan, magenta, and yellow pigments on paper, the mix won't absorb all the visible spectrum in a consistent manner. Our eyes are pretty sensitive to this, so we would see brown. This is why we add black ink to get pretty grays and blacks.

We call this approach to color CMYK (as opposed to RGB). If an artist is creating an image to be viewed on a screen, they will typically make an RGB image. If they are creating an image to be printed using pigments, they typically create a CMYK image. (Most of us don't care so much — we just let the computer do conversions between the two color spaces for us.)

Chapter 3

Images in Python

An image is usually represented as a three-dimensional array of 8-bit integers. NumPy arrays are the most commonly used library for this sort of data structure.

If you have an RGB image that is 480 pixels tall and 640 pixels wide, you will need a $480 \times 640 \times 3$ NumPy array.

There is a separate library (imageio) that:

- Reads an image file (like JPEG files) and creates a NumPy array.
- Writes a NumPy array to a file in standard image formats

Let's create a simply python program that creates a file containing an all-black image that is 640 pixels wide and 480 pixels tall. Create a file called `create_image.py`:

```
import NumPy as np
import imageio
import sys

# Check command-line arguments
if len(sys.argv) < 2:
    print(f"Usage {sys.argv[0]} <outfile>")
    sys.exit(1)

# Constants
IMAGE_WIDTH = 640
IMAGE_HEIGHT = 480

# Create an array of zeros
image = np.zeros((IMAGE_HEIGHT, IMAGE_WIDTH, 3), dtype=np.uint8)

# Write the array to the file
imageio.imwrite(sys.argv[1], image)
```

To run this, you will need to supply the name of the file you are trying to create. The extension (like .png or .jpeg) will tell imageio what format you want written. Run it now:

```
python3 create_image.py blackness.png
```

Open the image to confirm that it is 640 pixels wide, 480 pixels tall, and completely black.

3.1 Adding color

Now, let's walk through the image, pixel-by-pixel, adding some red. We will gradually increase the red from 0 on the left to 255 on the right.

```
import NumPy as np
import imageio
import sys

# Check command-line arguments
if len(sys.argv) < 2:
    print(f"Usage {sys.argv[0]} <outfile>")
    sys.exit(1)

# Constants
IMAGE_WIDTH = 640
IMAGE_HEIGHT = 480

# Create an array of zeros
image = np.zeros((IMAGE_HEIGHT, IMAGE_WIDTH, 3), dtype=np.uint8)

for col in range(IMAGE_WIDTH):

    # Red goes from 0 to 255 (left to right)
    r = int(col * 255.0 / IMAGE_WIDTH)

    # Update all the pixels in that column
    for row in range(IMAGE_HEIGHT):
        # Set the red pixel
        image[row, col, 0] = r

# Write the array to the file
imageio.imwrite(sys.argv[1], image)
```

When you run the function to create a new image, it will be a fade from black to red as you move from left to right:



Now, inside the inner loop, update the blue channel so that it goes from zero at the top to 255 at the bottom:

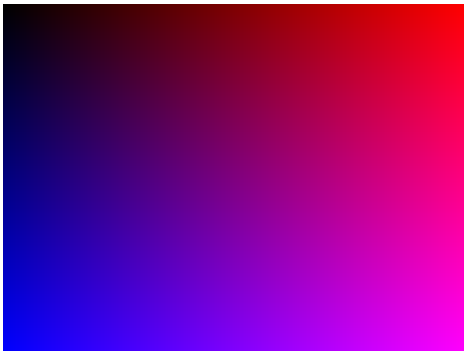
```
    # Update all the pixels in that column
    for row in range(IMAGE_HEIGHT):

        # Update the red channel
        image[row,col,0] = r

        # Blue goes from 0 to 255 (top to bottom)
        b = int(row * 255.0 / IMAGE_HEIGHT)
        image[row,col,2] = b

imageio.imwrite(sys.argv[1], image)
```

When you run the program again, you will see the color fades from black to blue as you go down the left side. As you go down the right side, the color fades from red to magenta.



Notice that red and blue with no green looks magenta to your eye.

Next, let's add some stripes of green:

```
import NumPy as np
import imageio
```

```
import sys

# Check command line arguments
if len(sys.argv) < 2:
    print(f"Usage {sys.argv[0]} <outfile>")
    sys.exit(1)

# Constants
IMAGE_WIDTH = 640
IMAGE_HEIGHT = 480
STRIPE_WIDTH = 40
pattern_width = STRIPE_WIDTH * 2

# Create an image of all zeros
image = np.zeros((IMAGE_HEIGHT, IMAGE_WIDTH, 3), dtype=np.uint8)

# Step from left to right
for col in range(IMAGE_WIDTH):

    # Red goes from 0 to 255 (left to right)
    r = int(col * 255.0 / IMAGE_WIDTH)

    # Should I add green to this column?
    should_green = col % pattern_width > STRIPE_WIDTH

    # Update all the pixels in that column
    for row in range(IMAGE_HEIGHT):

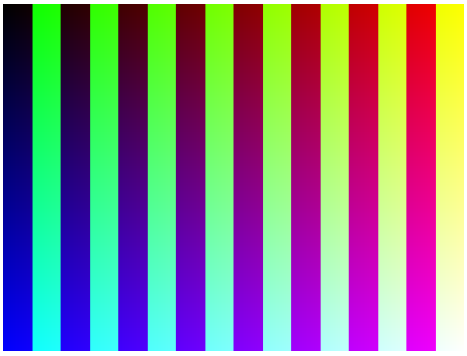
        # Update the red channel
        image[row,col,0] = r

        # Should I add green to this pixel?
        if should_green:
            image[row,col,1] = 255

        # Blue goes from 0 to 255 (top to bottom)
        b = int(row * 255.0 / IMAGE_HEIGHT)
        image[row,col,2] = b

imageio.imwrite(sys.argv[1], image)
```

When you run this version, you will see the previous image in half the stripes. In the other half, you will see that green fades to cyan down the left side, and yellow fades to white down the right side.



3.2 Using an existing image

imageio can also be used to read in any common image file format. Let's read in an image and save each of the red, green, and blue channels out as its own image.

Create a new file called `separate_image.py`:

```
import imageio
import sys
import os

# Check command line arguments
if len(sys.argv) < 2:
    print(f"Usage {sys.argv[0]} <infile>")
    sys.exit(1)

# Read the image
path = sys.argv[1]
image = imageio.imread(path)

# What is the filename?
filename = os.path.basename(path)

# What is the shape of the array?
original_shape = image.shape

# Log it
print(f"Shape of {filename}:{original_shape}")

# Names of the colors for the filenames
colors = ['red','green','blue']

# Step through each of the colors
for i in range(3):

    # Create a new image
    newimage = np.zeros(original_shape, dtype=np.uint8)
```

```
31
32      # Copy one channel
33      newimage[:,:,i] = image[:,:,i]
34
35      # Save to a file
36      new_filename = f"{colors[i]}_{filename}"
37      print(f"Writing {new_filename}")
38      imageio.imwrite(new_filename, newimage)
```

Now, you can run the program with any common RGB image type:

```
1  python3 separate_image.py dog.jpg
```

This will create three images: `red_dog.jpg`, `green_dog.jpg`, and `blue_dog.jpg`.

Chapter 4

Representing Natural Numbers

Natural numbers are positive whole numbers, such as 1, 2, 3, and so on. -5 is not a natural number. π is not a natural number. $\frac{1}{2}$ is not a natural number.

4.1 Base-10 (Decimal)

You are used to seeing the natural numbers represented in a **base-10** *Hindu-Arabic* or the *Decimal* numeral system. That is, when you see 2531 you think "2 thousands, 5 hundreds, 3 tens, and 1 one." Rewritten, this is:

$$2 \times 10^3 + 5 \times 10^2 + 3 \times 10^1 + 1 \times 10^0$$

The *radix* is the amount of number of unique values a base can have before increasing the amount of digits. For example, values 0, 1, 2, $\cdots$, 8, 9 are all the values for base-10. Once 1 is added to 9, we need to increase the amount of digits in a number.

In any Hindu-Arabic system, the location of the digits is meaningful: 101 is different from 110; the position of a number indicates its magnitude. Here are those numbers in Roman numerals: CI and CX. Roman numerals didn't have a symbol for zero at all.

The Hindu-Arabic system gave us really straightforward algorithms for addition and multiplication. For addition, you memorized the following table:

	0	1	2	3	4	5	6	7	8	9
0	0	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9	10
2	2	3	4	5	6	7	8	9	10	11
3	3	4	5	6	7	8	9	10	11	12
4	4	5	6	7	8	9	10	11	12	13
5	5	6	7	8	9	10	11	12	13	14
6	6	7	8	9	10	11	12	13	14	15
7	7	8	9	10	11	12	13	14	15	16
8	8	9	10	11	12	13	14	15	16	17
9	9	10	11	12	13	14	15 16	17	18	

When you multiplied two number together, you simply multiplied each pair of digits. 254×26 might look like this:

		2	5	4	
		×	2	6	
			2	4	6×4
		3	0		6×5
	1	2			6×2
			8		2×4
	1	0			2×5
+	4				2×2
6	6	0	4		

For multiplication, you memorized this table:

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7	8	9
2	0	2	4	6	8	10	12	14	16	18
3	0	3	6	9	12	15	18	21	24	27
4	0	4	8	12	16	20	24	28	32	36
5	0	5	10	15	20	25	30	35	40	45
6	0	6	12	18	24	30	36	42	48	54
7	0	7	14	21	28	35	42	49	56	63
9	0	9	18	27	36	45	54	63	72	81

4.2 Base-2 (Binary)

Binary, on the other hand, has only 2 digits it can have: 0 or 1. This kind of number system is often used for electrical and computer systems because the values can only be **on** or **off**.

Each digit in a binary number represents a power of 2, starting from the rightmost digit (the least significant bit, or LSB). For example:

$$\cdots + 2^3 + 2^2 + 2^1 + 2^0$$

For example,

$$1011_2 = (1 \cdot 2^3) + (0 \cdot 2^2) + (1 \cdot 2^1) + (1 \cdot 2^0) = 8 + 0 + 2 + 1 = 11_{10}$$

[1]

[1] The subscripts represent the radix or base.

Because the radix is 2, the numbers increase differently:

$$
\begin{aligned}
&0 : 0 \\
&1 : 1 \\
&2 : 10 \\
&3 : 11 \\
&4 : 100 \rightarrow (2^2 \cdot 1) + 0 + 0 = 4 \\
&5 : 101 \\
&6 : 110 \\
&7 : 111 \\
&8 : 1000 \\
&\dots \\
&15 : 1111 \cdots
\end{aligned}
$$

Conversion from Decimal to Binary

To convert 13_{10} to binary, repeatedly divide by 2 and record the remainders until the divisor reaches 0 or 1:

$13 \div 2 = 6$ remainder $16 \div 2 = 3$ remainder $03 \div 2 = 1$ remainder $11 \div 2 = 0$ remainder 1

Reading from bottom to top, $13_{10} = 1101_2$.

4.2.1 Bits and Bytes

A singular binary digit (0 or 1) is called a *bit*. A lightbulb, switch, or any sort of *Boolean* value (true or false) can be represented as a bit. A single binary digit is called a *bit*. Eight bits form a *byte*, which is a common unit of storage in computer systems:

$$1 \text{ byte} = 8 \text{ bits}$$

For example, the binary sequence 01001000 represents one byte of data.

We won't go into the depths of it here, but here are the basics of computer architecture. There are codes of 4-bit sequences that form up the *memory*. This can form the sections of it into different data types:

Common Data Types in Memory

Data Type	Size	Bit Length	Typical Use
Character	1 byte	8 bits	ASCII characters, small integers
Integer	4 bytes	32 bits	Whole numbers
Double	8 bytes	64 bits	Decimal (floating-point) values

Strings are formed by listing characters in consecutive memory addresses such that they are marked by a starting memory address and ending memory address.

8-bit binary numbers can be used to signify positive or negative numbers. The most significant bit (MSB) will alter sign, causing the range to be $[-2^7,\ 2^7 - 1]$. If the byte is negative (in terms of bits), invert each bit and add 1 to the inverted result. The other way is to multiply the MSB by negative one, and add until the value is reached, as seen below.

$$-85_{10} = -1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 10101011$$

Binary can be easy converted to *Hexadecimal*.

4.3 Base-16 (Hexadecimal)

Hexadecimal (from the latin *hexa* and *deca* for 16) is a representation of number with a radix of 16. Digits are represented through numeric numbers 0-9 and *A-F*.

Counting to 16 in hex goes as follows:

1
2
3
...
9
A (decimal value 10)
B (decimal value 11)
C (decimal value 12)
D (decimal value 13)
E (decimal value 14)
F (decimal value 15)

Converting between binary and hexadecimal is easier because the hex radix ($16 = 2^4$) is a power of the binary radix (2^1).

It takes 4 binary digits to represent 1 hex digit: 15 = F

For example,

$$0x47 = 0100\ 0111_2$$

$$0xBD = 1011\ 1101_2$$

4.3.1 Hex Color Codes

Hex colors are represented by 6 hexadecimal digits, grouped as two each for red, green, and blue (RGB). Each pair ranges from 00 (0 in decimal) to FF (255 in decimal), allowing 256 shades per color channel. For example:

$$\#FF0000 = \text{Red}, \quad \#00FF00 = \text{Green}, \quad \#0000FF = \text{Blue}$$

By mixing values, you can create millions of unique colors, e.g.:

$$\#FFA500 = \text{Orange}, \quad \#800080 = \text{Purple}, \quad \#FFFFFF = \text{White}, \quad \#000000 = \text{Black}$$

There are $256^3 = 16{,}777{,}216$ possible color combinations in the RGB hex system. The higher the red, green, or blue values are (closer to 255), the brighter the component becomes and the overall color shifts closer to white. Conversely, the lower the values are (closer to 0), the darker the component becomes and the overall color shifts closer to black.

Included in your digital resources are multiple hex videos, as well as an interactive hex color guessing game.

4.4 Base-8 (Octal)

Octal is another representation of radix 8. Digits range from 0 to 7, and the number of digits increase the threshold is passed.

Just as in the above systems, octal increases by the radix —– powers of 8. For example,

$$135_8 = (1 \times 8^2) + (3 \times 8^1) + (5 \times 8^0) = 64 + 24 + 5 = 93_{10}$$

4.4.1 Why Octal Matters

Octal was historically important in computing because early computers often worked with word sizes that were multiples of 3 bits (such as 12, 24, or 36). Since each octal digit corresponds exactly to 3 binary digits, it was a convenient shorthand for binary values before hexadecimal became more widespread. For example:

$$110101_2 = 65_8$$

Even though modern systems mostly use hexadecimal, octal is still used in some contexts, such as Unix file permissions (e.g., `chmod 755`).

Chapter 5

Floating Point Arithmetic

5.1 Why Computers Can't Count Perfectly

Computers are known to be very precise. Modern processors can perform billions of calculations per second, so it's not intuitive to realize they have this persistent limitation: they cannot store most numbers exactly.

Let us think about what it would actually take to store a number like $\frac{1}{3}$. You probably know its decimal expansion:

$$\frac{1}{3} = 0.3333333\ldots$$

The digits go on forever. Now imagine you are a computer with a fixed amount of memory. You cannot store infinitely many digits. So you store 0.333 or 0.3333333, depending on how much space you have, and you accept that the result is not exactly right. That gap between the number you wanted and the number you actually stored is called *approximation error*, and it never fully goes away.

This is not just a problem for fractions with repeating decimals. Irrational numbers like $\sqrt{2} = 1.41421356\ldots$ and transcendental numbers like $\pi = 3.14159265\ldots$ also have infinite, non-repeating decimal expansions. Every one of them must be approximated when stored in a computer.

To handle this systematically, mathematicians and engineers developed the *floating-point system*. Before we look at how it works, it helps to understand the two core reasons why any finite system will always fall short of the real number line.

5.1.1 The Number Line is Continuous and Unbounded

Between any two real numbers, no matter how close together, there are infinitely many others. Pick any two decimals you like. For example, if you picked 1.00 and 1.01, you can always find a number between them: 1.005, 1.0051, 1.00500001, and so on without end. This is referred to as *continuity*. A computer's number system cannot be continuous. It stores numbers using a fixed string of digits, which means it can only represent finitely many values. Real numbers have to be rounded to the nearest representable value.

The real numbers stretch infinitely in both directions. There is no largest real number and no smallest positive real number. A computer's memory is finite, so its number system must have both a largest and a smallest representable value. Numbers too large to

store cause a problem called *overflow*. Numbers too small to distinguish from zero cause *underflow*.

Exercise 3 Overflow and Underflow

Suppose a floating-point system can only represent numbers between 10^{-4} and 10^{8}.

1. Planck's constant is approximately 6.63×10^{-34} J·s. Does it cause underflow, overflow, or neither in this system?
2. The speed of light is approximately 3.00×10^{8} m/s. Does it cause underflow, overflow, or neither in this system?
3. Give one example of a number that would cause overflow.

Working Space

Answer on Page 43

So what can we do?

We cannot fix these limitations, because they are baked into the nature of finite computation. What we can do is design the floating-point system carefully so that the errors it introduces are small, predictable, and manageable. That is exactly what engineers and mathematicians have spent decades working out, and it is what the rest of this chapter is about!

The key insight is that a floating-point number is not just a string of digits. A floating-point number has three parts: a *sign*, a *significand* (sometimes called the mantissa), and an *exponent*.

5.2 Binary and the Floating-Point System

In the previous section you saw that computers cannot store most numbers exactly. Now the natural question is: how do they store numbers at all? The answer starts at the hardware level, with the smallest possible piece of information a computer can hold.

5.2.1 Binary System

Every process inside a CPU comes down to the flow of electrical current through tiny switches called *transistors*. A transistor is either on (carrying current) or off (not carrying current). That gives us exactly two states, which we write as 0 and 1. This is why computers use *binary*: it is the only number system that hardware can physically implement with a simple on/off switch.

A single binary digit is called a *bit*. Eight bits grouped together form a *byte*. You have probably seen units like megabyte (MB) or gigabyte (GB) before. One megabyte is 10^6 bytes, and one gigabyte is 10^9 bytes. A modern smartphone holds tens of billions of bits.

Exercise 4 Bits and bytes

Working Space

1. A file is 4 megabytes. How many bytes is that? How many bits?
2. A hard drive stores 500 gigabytes. How many bits can it hold? Write your answer using scientific notation.
3. A single bit can be 0 or 1. Two bits together can be 00, 01, 10, or 11, which are four possible values. How many distinct values can 8 bits (one byte) represent? How about 64 bits?

Answer on Page 43

5.2.2 Floating-Point Number

Recall that a floating-point number has three parts: a *sign*, a *significand* (sometimes called the mantissa), and an *exponent*.

Before we write any formulas, let us see why all three parts are necessary with a base-10 analogy.

Consider the number -0.001234. Written in scientific notation it is -1.234×10^{-3}. Three pieces of information do all the work:

- The **sign** $(-)$ tells us the number is negative.
- The **significand** (1.234) holds the meaningful digits.
- The **exponent** (-3) tells us how large or small the number is by shifting the decimal point.

Now watch what the exponent does when we fix the significand at 0.1234 and vary it in base 10:

Float	Value
0.1234×10^{-2}	0.001234
0.1234×10^{1}	1.234
0.1234×10^{4}	1234
0.1234×10^{8}	12,340,000

The decimal point *floats*, which means it can sit anywhere, controlled entirely by the exponent. That is why these are called floating-point numbers. The significand always carries the same digits; only the scale changes.

Exercise 5 The floating point in action

The significand is fixed at 0.5050 and the base is 10. Fill in the value represented by each float.

Float	Value
0.5050×10^{0}	
0.5050×10^{2}	
0.5050×10^{-1}	
0.5050×10^{5}	

Now answer: if you want to represent 505,000,000 using the same significand, what exponent do you need?

Working Space

Answer on Page 43

5.2.3 Floating-Point System

A floating-point system is defined by four numbers, written (β, p, m, M). Every float in that system looks like this:

$$\pm\,(0.d_1\,d_2\,\cdots\,d_p)_\beta\ \times\ \beta^e$$

This looks intimidating at first, but each symbol has a simple job. Let us unpack them one at a time.

β represents the *base*. This is the number that the exponent is a power of. In everyday life we use $\beta = 10$. Computers use $\beta = 2$ (binary).

p represents the *precision*. This is how many digits the significand gets to use. The significand $0.d_1d_2\cdots d_p$ has exactly p digits, each chosen from $\{0, 1, \ldots, \beta - 1\}$. More digits means more precision, that is, you can represent numbers more accurately. Fewer digits means faster arithmetic but more rounding error.

e represents the *exponent*. This shifts the significand left or right. The exponent is not free to be any integer; it is capped between a minimum m and a maximum M. Those bounds set the range of the system, i.e., how large and how small the representable numbers can be.

m and M represents the *exponent bounds*. The exponent must satisfy $m \leq e \leq M$. Together they control overflow and underflow. If a result needs an exponent below m, underflow occurs. If it needs an exponent above M, overflow occurs.

We say the system is *parametrized* by (β, p, m, M). Changing any one of the four gives a new floating-point system with different capabilities and different limitations.

Exercise 6 Reading the parameters

A toy floating-point system has parameters $(\beta, p, m, M) = (10, 4, -3, 4)$.

Working Space

1. What base does this system use?
2. How many significant digits does it carry?
3. What are the smallest and largest allowed exponents?
4. Can this system represent 12,345? Try writing 12,345 in the form $0.d_1d_2d_3d_4 \times 10^e$ and check whether the exponent is within bounds.
5. Can it represent 0.00001? Show your reasoning.
6. Write two numbers that *can* be represented exactly, and one that causes overflow.

Answer on Page 44

5.2.4 Connecting to Binary Floating-Point

All of the above applies to a computer's binary floating-point system, with $\beta = 2$ instead of $\beta = 10$. The significand digits $d_1, d_2, \ldots, d_p$ are now bits: each one is either 0 or 1. The exponent is a power of 2.

So a binary float looks like:

$$\pm\,(0.d_1\,d_2\,\cdots\,d_p)_2 \;\times\; 2^e, \qquad d_i \in \{0, 1\}, \quad m \le e \le M$$

The specific choices of p, m, and M define how much memory is used and how accurately numbers are stored. The international standard that fixes these choices for every modern computer is called *IEEE 754*, and it is the subject of the next section.

5.3 The IEEE 754 Standard

In 1985, after years of debate, the Institute of Electrical and Electronics Engineers published the *IEEE 754 standard*. It was accepted internationally in 1989, and it is still the standard used in virtually every computer, phone, and calculator you use today.

5.3.1 Floats

If we are going to store a float in a fixed number of bits, how do we split those bits among the three parts: sign, significand, and exponent?

For the sign, there are only two options, positive and negative, so a single bit handles it: 0 for positive, 1 for negative.

That leaves $N-1$ bits to split between the significand and the exponent. IEEE 754 defines two standard ways to do this split, called *single precision* and *double precision*.

	Total bits	Significand bits (p)	Exponent bits
Single precision	32	23	8
Double precision	64	52	11

In both cases the layout is the same. The sign occupies the first bit, the significand fills the next p bits, and the exponent takes the remaining bits. Double precision uses twice as many bits in total, which gives it both greater precision and a wider range of representable exponents.

For single precision, the 8 exponent bits can represent integers up to $M = 127$, so the largest float in single precision is roughly 10^{38}. Double precision is the standard used in modern computers.

Exercise 7 Bit Allocation

A floating-point system uses $N = 16$ bits total. The sign takes 1 bit and the significand takes 10 bits.

1. How many bits are left for the exponent?
2. If those exponent bits can represent integers from -31 to 32, what is the largest exponent M?
3. Would you expect this system to be more or less precise than single precision? Why?

Working Space

Answer on Page 44

5.3.2 Machine Epsilon

The *machine epsilon* is commonly written as u. It measures the largest relative gap between a real number and its nearest float. In other words, it tells you the worst-case precision of your system.

For a system that rounds to the nearest float, machine epsilon is

$$u = \frac{1}{2}\beta^{1-p},$$

where β is the base and p is the precision. For IEEE 754 double precision, $\beta = 2$ and $p = 52$, giving $u \approx 2.22 \times 10^{-16}$. That means any real number is stored with a relative error no worse than about 2×10^{-16} [correct punctuation] roughly 16 significant decimal digits of accuracy.

If you have access to Python, you can check your machine epsilon directly:

```
import sys
print(sys.float_info.epsilon)   # double precision machine epsilon
```

Every machine running IEEE 754 double precision should return the same value: `2.220446049250313e-16`.

Exercise 8 Machine Epsilon

A base-10 floating-point system has precision $p = 6$ and uses rounding.

1. Compute machine epsilon u for this system.
2. A real number x is stored as a float $\tilde{x}$. Using your answer above, what is the worst-case relative error $\frac{|\tilde{x}-x|}{|x|}$?
3. If p were doubled to $p = 12$, how would u change?

Working Space

Answer on Page 44

Good to know: IEEE 754 has been updated since 1985. The current standard includes *quadruple precision* (128 bits) for applications that need the highest accuracy. This includes physics simulations and cryptographic computations. It can also define integer data types, `int8`, `int32`, and `int64`. These are used in situations where you only need whole numbers and want to avoid floating-point arithmetic entirely.

The existence of multiple precisions is a reminder that there is always a trade-off. More bits means more accuracy and a wider range, but also more memory and slower computation. Choosing the right precision for a given problem is the real engineering decision.

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 7)

The two triangles are similar; one is 2 m and 3m, the other is x cm and 3 cm.

The image of the cow is 2 cm tall.

Answer to Exercise 2 (on page 9)

The pixels will remain dark, or black, because the energy of the incoming photons is less than the work function of the sensor material. Calculating $E = \frac{1240}{1200} = 1.033\,\text{eV}$, which is less than the work function of 1.1 eV.

Answer to Exercise 3 (on page 34)

1. Underflow. 6.63×10^{-34} is far below 10^{-4}.
2. Neither. 3.00×10^{8} is exactly at the upper boundary.
3. Any number greater than 10^{8}.

Answer to Exercise 4 (on page 35)

1. 4×10^{6} bytes; $4 \times 10^{6} \times 8 = 3.2 \times 10^{7}$ bits.
2. $500 \times 10^{9} \times 8 = 4 \times 10^{12}$ bits.
3. $2^{8} = 256$ values for one byte; $2^{64} \approx 1.8 \times 10^{19}$ values for 64 bits.

Answer to Exercise 5 (on page 36)

- $0.5050 \times 10^0 = 0.5050$
- $0.5050 \times 10^2 = 50.50$
- $0.5050 \times 10^{-1} = 0.05050$
- $0.5050 \times 10^5 = 50{,}500$

For $505{,}000{,}000 = 0.5050 \times 10^9$, the exponent needed is 9.

Answer to Exercise 6 (on page 38)

1. Base 10.
2. 4 significant digits.
3. Smallest: $m = -3$; largest: $M = 4$.
4. $12{,}345 = 0.12345 \times 10^5$. The exponent 5 exceeds $M = 4$, so this causes **overflow**. The system would need to round to $0.1235 \times 10^5 = 12{,}350$ and even then the exponent is out of range.
5. $0.00001 = 0.1000 \times 10^{-4}$. The exponent -4 is below $m = -3$, so this causes **underflow**.
6. Any number in the range 10^{-3} to $(1 - 10^{-4}) \times 10^4$ works, for example $0.5000 \times 10^2 = 50.00$ or $0.9999 \times 10^4 = 9999$. Overflow example: any number with magnitude above 9999, such as 100,000.

Answer to Exercise 7 (on page 40)

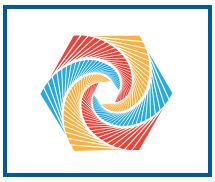
1. $16 - 1 - 10 = 5$ bits for the exponent.
2. $M = 32$.
3. Less precise than single precision. Single precision has $p = 23$ significand bits; this system has only $p = 10$, so it can store fewer digits and will produce larger roundoff errors.

Answer to Exercise 8 (on page 41)

1. $u = \frac{1}{2} \times 10^{1-6} = \frac{1}{2} \times 10^{-5} = 5 \times 10^{-6}$.

2. The worst-case relative error is $u = 5 \times 10^{-6}$, meaning about 6 significant digits of accuracy.

3. $u = \frac{1}{2} \times 10^{1-12} = 5 \times 10^{-12}$. Doubling the precision shrinks machine epsilon by a factor of 10^6.



Index

A-F, 30
approximation error, 33
aqueous humor, 14
aspect ratio, 10
astigmatism, 16

base, 37
base-2, 28
binary, 28, 35
    counting in, 29
bit, 29, 35
Boolean, 29
byte, 29, 35

camera, 3
camera
    lens, 7
    lens
        focal length, 8
    pinhole, 6
cataract, 15
cmyk, 19
color, 16
computer architecture, 29
continuity, 33
cornea, 13

Decimal, 27
double precision, 39

exponent, 34, 35, 37
exponent bounds, 37

eye, 13
    astigmatism, 15
    cataract, 15
    farsightedness, 15
    glaucoma, 14
    iris, 13
    myopia, 15
    nearsightedness, 15
    pupil, 13
    retina, 14
    sclera, 13

farsighted, 16
floating-point system, 33
focal length, 8
focal point, 8

glaucoma, 15

hex colors, 31
Hexadecimal, 30
hexadecimal, 30, 31
Hindu-Arabic, 27
hyperopic, 16

IEEE 754, 38
IEEE 754 standard, 39
iris, 13

lens, 7

machine epsilon, 40

memory, 29
myopic, 16

nearsighted, 16

Octal, 31
overflow, 34

parametrized, 37
photon
    colors of, 5
    wavelength, 5
precision, 37
pupil, 13

quadruple precision, 41

radix, 27
resolution, 9
retina, 14
rgb, 9, 18

sclera, 14
sign, 34, 35
signed bits, 30
significand, 34, 35
single precision, 39

transistors, 35

underflow, 34